UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS INFORMÁTICOS

Actividad Aprendizaje Profundo

Asignatura: Aprendizaje Automático II

18/3/2026

Grupo 6

Pablo Tsang (pablo.tsang@alumnos.upm.es)

Alejandro Escorial (alejandro.escorial@alumnos.upm.es)

Oliver Arnaldo Anderson (oliver.anderson@alumnos.upm.es)

Diego Mateo (diego.mateofer@alumnos.upm.es)

Google Colab

TABLA DE CONTENIDOS

1. Introducción 2

2. Proceso de diseño y experimentación 2

- **2.1. Modelo inicial** 2
- **2.2. Regularización** 3
- **2.3. Optimizadores** 4
- **2.4. Arquitectura de la red** 7
- **2.5. Últimos ajustes** 7

3. Resultados finales 9

4. Conclusiones 12

5. Referencias 13

1. Introducción

En este trabajo se van a aplicar y comparar distintas técnicas de aprendizaje profundo (Deep Learning) vistas en la asignatura mediante el uso de la librería Keras de TensorFlow en Python sobre los datos de FIFA 19 complete player dataset, siendo el objetivo estimar la calidad global de los distintos jugadores de fútbol y clasificarlos en *Poor*, *Intermediate*, *Good* or *Excellent* según corresponda.

El conjunto de datos original contiene 18207 instancias de jugadores y 89 variables de cada uno, pero para llevar a cabo la clasificación adecuadamente primero se aplica un preprocesado dado en la asignatura que lo divide en dos datasets, el de la variable objetivo *Overall* con la calidad global y el de las variables predictoras. En ambos datasets se quedan 16134 instancias tras eliminar a los porteros e instancias con valores faltantes, pero cada uno recibe un tratado distinto extra.

En el dataset de variables predictoras se quedan 22 tras quitar variables irrelevantes para el problema como el número de camiseta o con correlación menor a 0.3 entre otros. Además, se aplica un escalado de datos Min-Max para asegurarse de que estén en el rango [-1, 1] y tengan igual importancia en la red.

Por otro lado, en el dataset con la variable objetivo se discretiza dicha variable en sus cuartiles según *Poor* - [46, 62], *Intermediate* - [63, 66], *Good* - [67, 71] y *Excellent* - [72, 94], para después aplicar la técnica One-Hot para que todos los datos sean numéricos.

En resumen, se va a partir de dos datasets derivados del conjunto de datos original, uno con las variables predictoras relevantes y escaladas y otro con la variable objetivo discretizada y codificada con One-Hot de modo que a la hora de construir las futuras redes neuronales de clasificación no haya ningún problema.

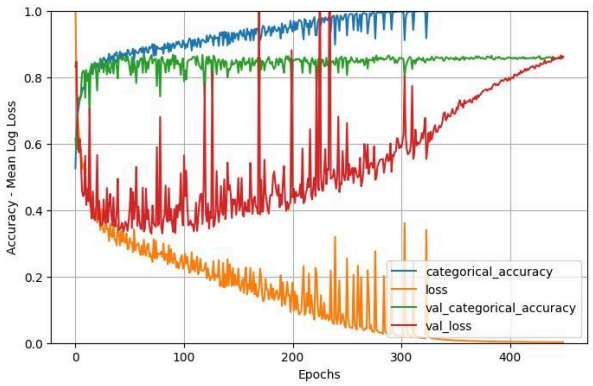
2. Proceso de diseño y experimentación

Para encontrar el mejor modelo hemos seguido una estrategia iterativa en la que, partiendo de un modelo inicial, aplicamos optimizadores para reducir el bias y posteriormente métodos de regularización para disminuir la varianza y evitar el sobreajuste. Como estas métricas suelen estar inversamente relacionadas, en el caso en el que el sesgo volviese a aumentar al aplicar la regularización, repetiremos el ciclo hasta encontrar el modelo óptimo.

2.1. Modelo inicial

Como punto de partida decidimos inicializar los pesos utilizando HeNormal ya que usamos una función de activación ReLU. Según pudimos observar, los resultados con esta inicialización eran muy superiores frente a iniciaciones aleatorias, por lo que decidimos mantenerla como inicialización del modelo.

Con los valores por defecto y probando con 4 capas ocultas con 300, 150, 75 y 30 neuronas obtuvimos los siguientes resultados:



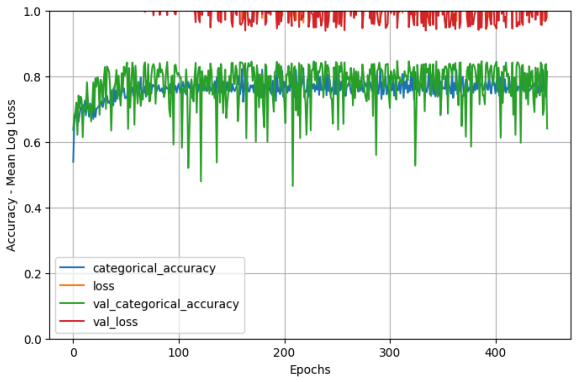
Este modelo inicial presenta un claro problema de overfitting ya que el accuracy en Train llega prácticamente a 1 mientras que en validación se estabiliza en torno al 0.85. Además, si nos fijamos en las pérdidas (loss), observamos como la diferencia se hace cada vez mayor.

Por lo tanto, como nuestro principal problema está relacionado con la alta varianza y no tanto con el bias, decidimos probar primero diferentes métodos de regularización.

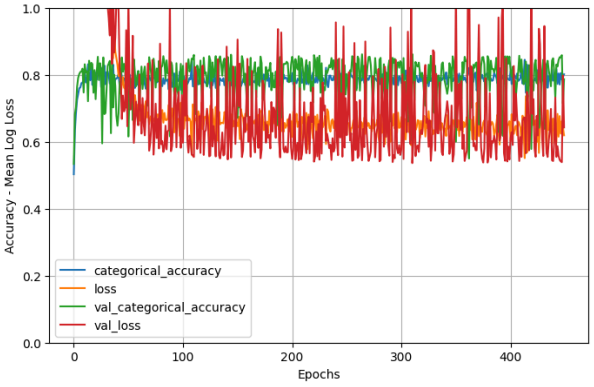
2.2. Regularización

En el proceso de regularización hemos probado una gran cantidad de combinaciones de parámetros para poder determinar la mejor solución para este problema. Lo primero que hicimos fue decidir si usar la regularización **L1**, o por el contrario la **L2**. Para ello realizamos varias pruebas con distintos parámetros de regularización tanto de L1 como de L2, hasta confirmar que los mejores los obtuvimos cuando usábamos L2 a 0.01 de regularización.

Regularización L1 (0.01)



Regularización L2 (0.01)



Lo siguiente era comparar este método con otros dados en la asignatura, por lo que decidimos compararlo con un **dropout**. Además, para nosotros este método era de especial interés, ya que ayuda a la simplificación de la arquitectura de la red.

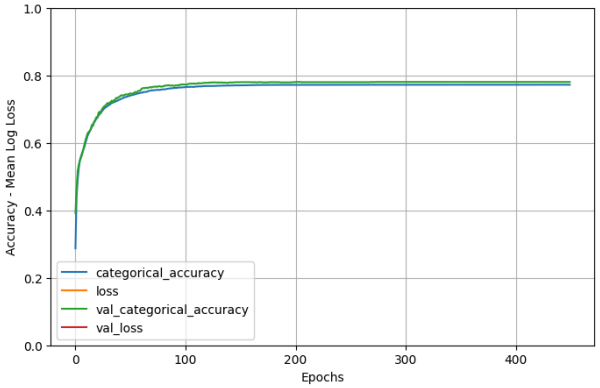
Tras probar varios dropout rates, observamos que dropout daba resultados muy similares a la regularización L2 con un dropout rate de 0.1 por capa.

Por último, intentamos combinar ambos métodos, pero daba peores resultados, así que decidimos quedarnos únicamente con L2.

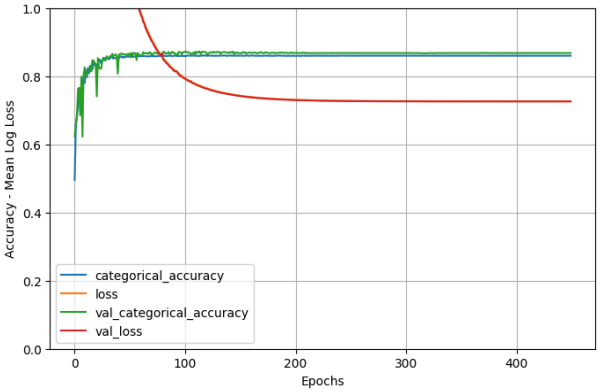
2.3. Optimizadores

Una vez efectuada la regularización, nuestro objetivo fue reducir el error de bias, por lo que realizamos un estudio para tratar de encontrar el mejor algoritmo de optimización, capaz de minimizar el error en el menor tiempo posible y sin caer en óptimos locales.

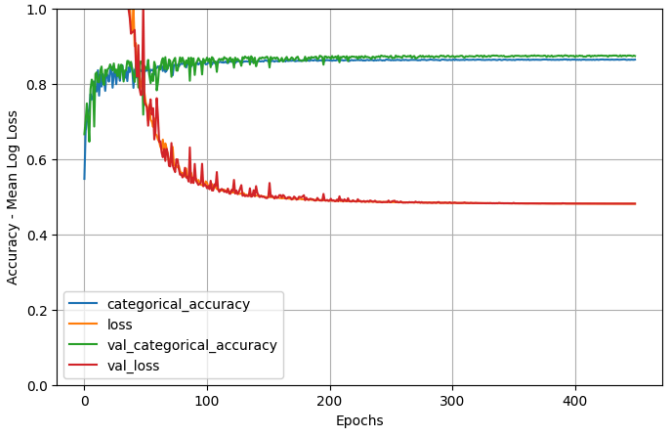
Comenzamos probando un modelo base con **Learning Rate Decay** (decaimiento escalonado cada 50 steps). En un principio probamos con un decay rate de 0.96 y un learning rate inicial no demasiado alto, obteniendo resultados bastante mejorables (en torno al 0.79 de accuracy y con un loss muy elevado).



Como posiblemente el modelo estaba cayendo en mínimos locales, decidimos ir aumentando progresivamente la tasa de aprendizaje inicial hasta obtener mejores resultados. Con un learning rate inicial alto de 0.1, el accuracy se estabilizó en torno al 0.85 y la pérdida bajó a 0.7.



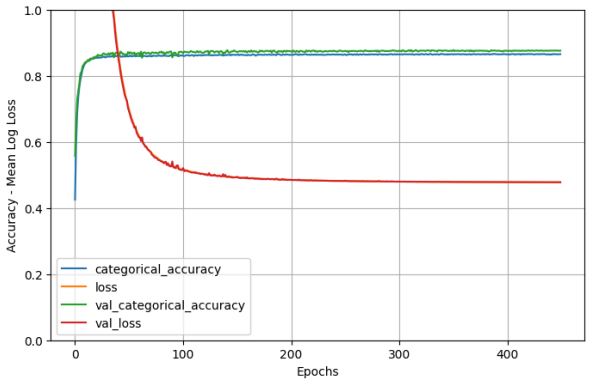
Sin embargo, como el loss seguía siendo alto, decidimos modificar el decay rate en busca de mejores resultados, considerando finalmente como mejor configuración un learning rate inicial de 0.01 y un decay_rate de 0.98.



$Accuracy \rightarrow 0.865$
$Loss \rightarrow 0.479$
$Val_Accuracy \rightarrow 0.877$
$Val_Loss \rightarrow 0.479$

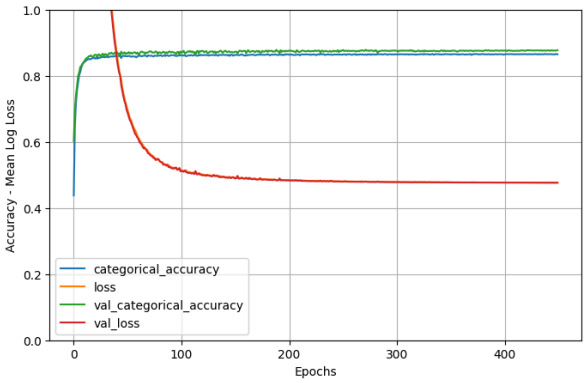
A continuación decidimos probar los algoritmos de **Momentum** y **Nesterov**. En general, nos dimos cuenta de que los resultados eran bastante peores dejando un learning rate fijo, por lo que decidimos combinar estos optimizadores con el Learning Rate Decay implementado anteriormente. Fusionar ambos algoritmos nos permitió obtener resultados tan buenos como los anteriores pero reduciendo las oscilaciones.

Nesterov=True



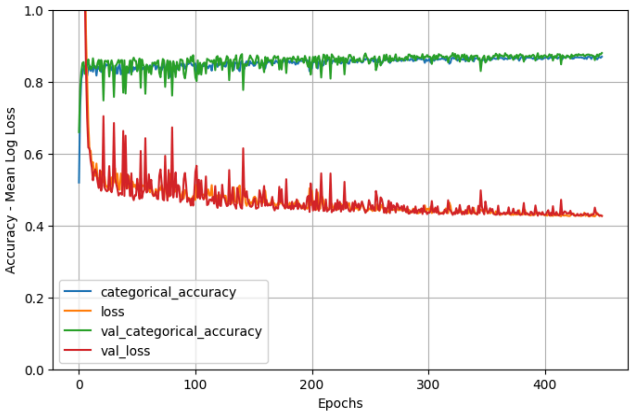
	categorical_accuracy	loss	val_categorical_accuracy	val_loss
449	0.865344	0.477472	0.876007	0.478072

Nesterov=False



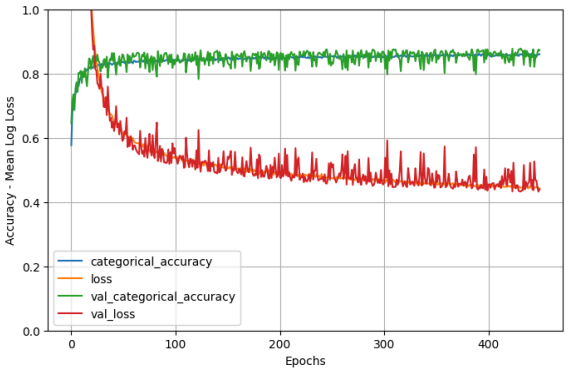
	categorical_accuracy	loss	val_categorical_accuracy	val_loss
449	0.86488	0.476645	0.877247	0.476505

Para poder disminuir el loss probamos a aumentar el número de steps en el algoritmo de decay y nos dimos cuenta de que a medida que aumentamos el número de pasos (hasta unos 200), menor era la pérdida pero mayores oscilaciones se producían ya que el learning rate tardaba más en disminuir su valor. Algo muy similar ocurría al aumentar el valor del learning rate inicial. En busca de un equilibrio, concluimos que la mejor configuración para el algoritmo de optimización Momentum es combinarlo con Learning Rate Decay con 100 steps y 0.05 de valor inicial. También comprobamos que para nuestro caso utilizar Nesterov no proporcionaba ninguna ventaja.



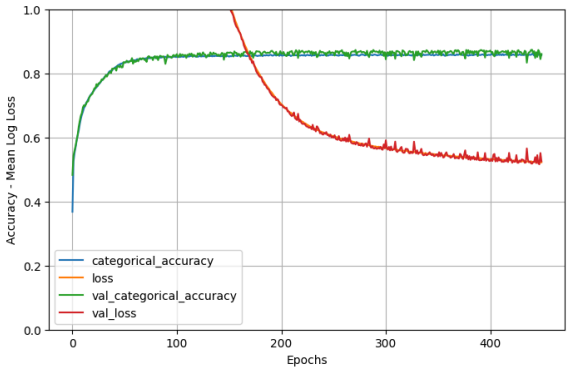
$Accuracy \rightarrow 0.869$
$Loss \rightarrow 0.427$
$Val_Accuracy \rightarrow 0.879$
$Val_Loss \rightarrow 0.426$

Posteriormente probamos **RMSProp** con un learning rate de 0.001 pero los resultados no mejoraron a los anteriores:

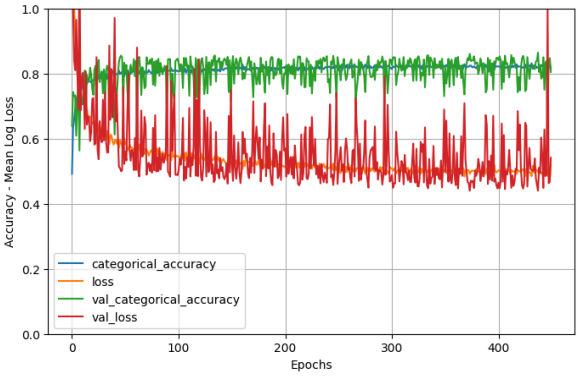


Además, tanto aumentar como disminuir el valor de la tasa de aprendizaje empeoraba enormemente las métricas.

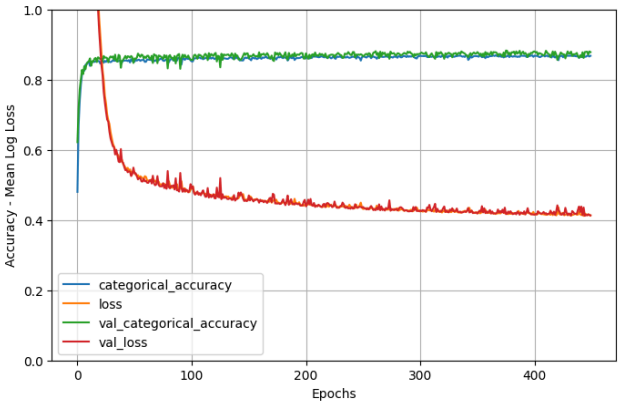
Lr=0.0001



Lr=0.01



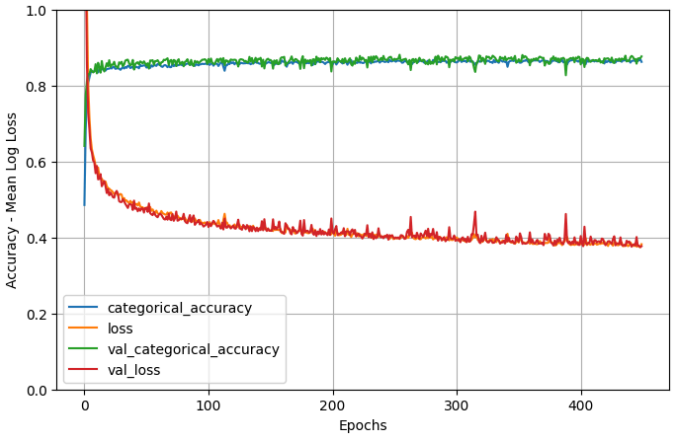
Por último, decidimos probar el optimizador **Adam**, que con un learning rate de 0.001 demostró ser el más óptimo:



$Accuracy \rightarrow 0.868$
$Loss \rightarrow 0.413$
$Val_Accuracy \rightarrow 0.878$
$Val_Loss \rightarrow 0.413$

2.4. Arquitectura de la red

Como después de estos cambios no teníamos grandes problemas de bias, nos propusimos reducir la complejidad de la red. Lo primero que hicimos fue reducir el número de neuronas en cada capa y, como las métricas no empeoraron, decidimos reducir el número de capas a 3, con 256, 128 y 64 neuronas. Finalmente, la arquitectura que nos resultó más simple y efectiva para este problema fue 64, 32 y 16 neuronas. Esta configuración no empeoraba las métricas obtenidas con los modelos anteriores pero reducir aún más la arquitectura de la red comenzaba a dar problemas. Esta simplificación de la arquitectura produjo, de hecho, una ligera mejora en las métricas:

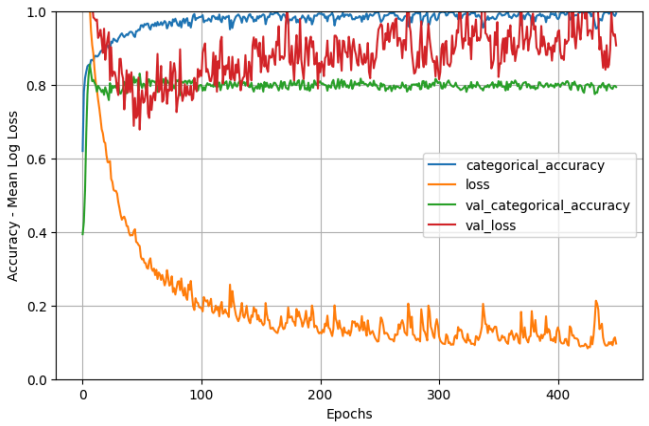


$Accuracy \rightarrow 0.862$
$Loss \rightarrow 0.382$
$Val_Accuracy \rightarrow 0.877$
$Val_Loss \rightarrow 0.377$

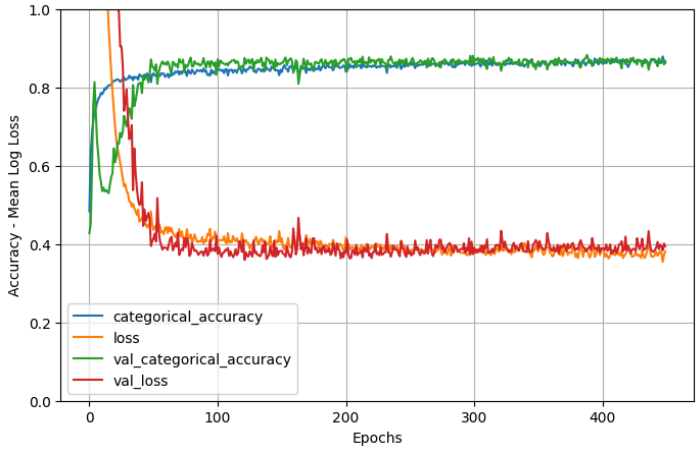
2.5. Últimos ajustes

Tras acabar con el proceso principal, decidimos realizar ajustes complementarios. Lo primero que implementamos fue el **early stopping**. Para ello decidimos que si el validation loss no mejoraba en 20 épocas consecutivas, frenarían el entrenamiento.

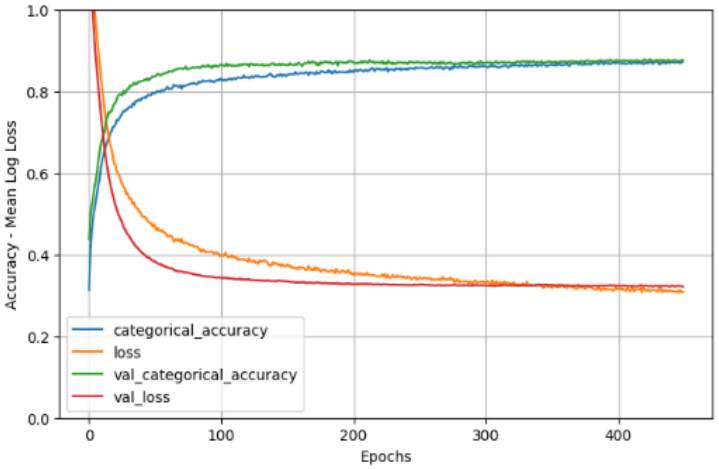
También decidimos probar **batch normalization** pero el primer resultado no fue nada bueno:



Modificando ligeramente los parámetros (utilizar dropout a 0.3) solucionamos el problema pero como no obtuvimos una mejora significativa en las métricas volvimos al modelo anterior.

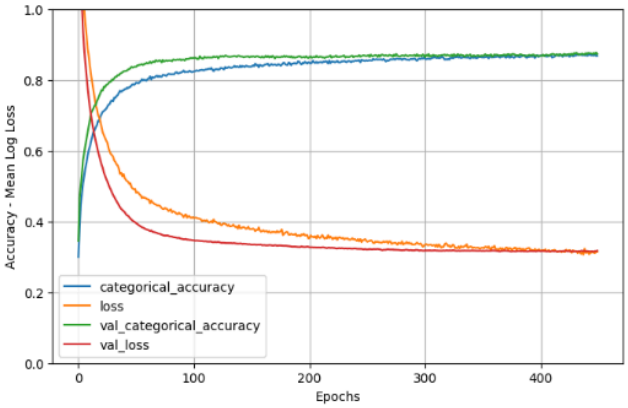


Además, aunque no fuera uno de los optimizadores principales vistos en la asignatura, decidimos probar **Nadam** ya que sí se mencionaba como opción. Probamos este optimizador con dropout (rate de 0.1 en cada capa) y con distintos learning rate hasta obtener una combinación (learning rate de 0.0004) que aportaba mejores resultados que los obtenidos hasta ahora con Adam:



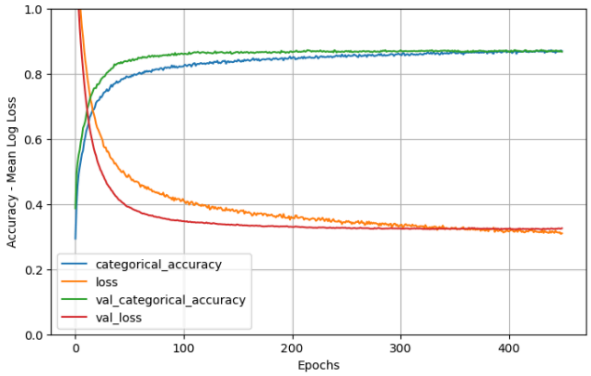
$Accuracy \rightarrow 0.873$
$Loss \rightarrow 0.308$
$Val_Accuracy \rightarrow 0.877$
$Val_Loss \rightarrow 0.321$

A continuación, probamos a cambiar los dropout rates por capas en lugar de todos iguales y, tras varias combinaciones, observamos que rates de 0.1, 0.2 y 0.1 para la primera, segunda y tercera capa respectivamente también mejoraron los resultados. Esto tiene sentido ya que la segunda capa es la que solía acumular mayor peso.

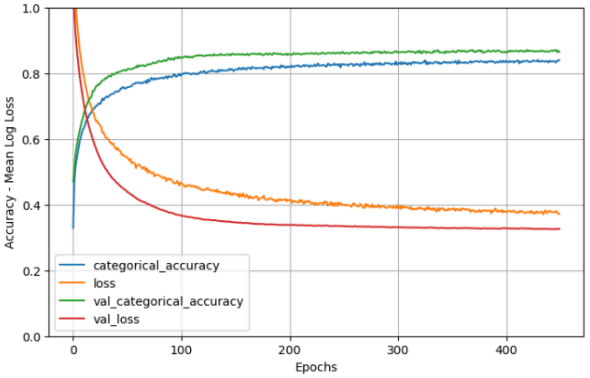


También probamos otras funciones de activación vistas derivadas de **ReLU** como **ELU** o **Leaky ReLU** para darnos cuenta de que no aportaban mejores resultados.

Leaky ReLU



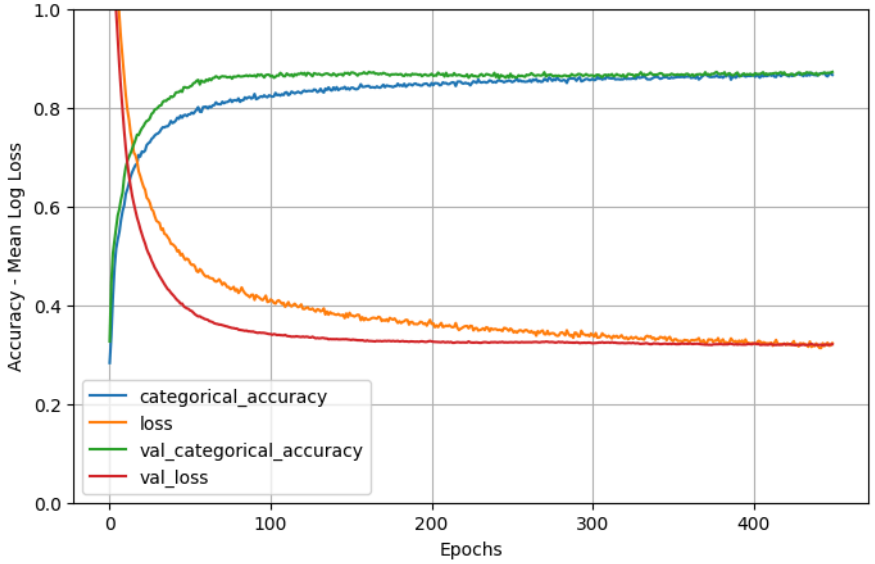
ELU



3. Resultados finales

Al final, hemos obtenido como mejor modelo el siguiente:

- Batch size = 512
- Learning rate = 0.0004
- Función de activación → ReLU
- Red de tres capas con 64, 32 y 16 neuronas.
- Regularización → Dropout con rates de 0.1, 0.2 y 0.1 para primera, segunda y tercera capa respectivamente.
- Algoritmo de optimización → Nadam



En la gráfica se observa que se alcanza una accuracy de 0.866 y una pérdida de 0.32 en el conjunto de entrenamiento, mientras que en el conjunto de validación se obtiene una accuracy de 0.872 y una pérdida de 0.32. Teniendo en cuenta un error bayesiano del 10%, estos resultados pueden considerarse muy favorables.

La pérdida media de 0.32 puede interpretarse mediante $e^{-0.32}$ (0.726), que corresponde a la media geométrica de la probabilidad asignada a la clase correcta. Este valor es muy superior a 0.25, que sería el esperado en un modelo que predice aleatoriamente entre las cuatro clases, lo que indica que el modelo realiza predicciones significativamente más informadas que el azar.

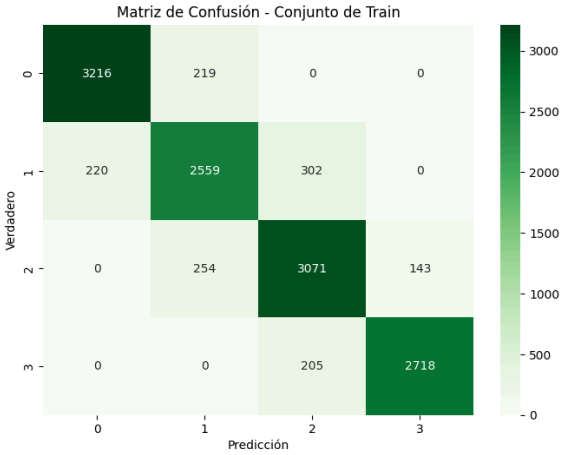
También hemos realizado un análisis de las pérdidas de cada clase, dando lugar los siguientes resultados:

	Poor	*Intermediate*	*Good*	*Excellent*
Entrenamiento	0.127	0.380	0.328	0.151
Validación	0.175	0.529	0.398	0.184

Las clases situadas en los extremos presentan una menor pérdida, mientras que aquellas con un mayor número de clases adyacentes muestran una mayor pérdida. Esto se debe a la dificultad de trazar fronteras de decisión claras entre clases cercanas sin comprometer la capacidad de generalización del modelo.

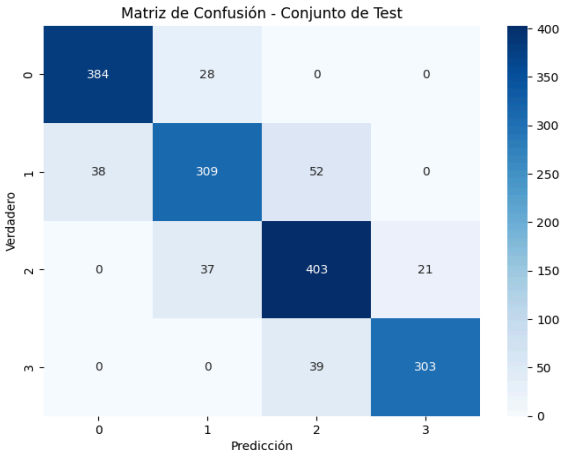
Por otro lado, en el modelo se observa que tanto el *accuracy* como la pérdida del conjunto de validación convergen antes que los del conjunto de entrenamiento. Este comportamiento puede explicarse por el uso de dropout, que introduce ruido durante el entrenamiento al desactivar aleatoriamente neuronas, lo que hace que las métricas de entrenamiento evolucionen de forma más irregular. Además, el uso de mini-batches introduce cierta variabilidad adicional durante el entrenamiento, mientras que la evaluación en validación es más estable.

Otra cuestión que cabe destacar es que el modelo arroja mejores resultados con una arquitectura neuronal más sencilla. En comparación con la arquitectura inicial, se ha reducido una capa y el número total de neuronas ha pasado de 555 a 112. Este comportamiento puede relacionarse con el concepto de "*Lottery Ticket Hypothesis*" [1], que sugiere que dentro de redes grandes existen subredes más pequeñas capaces de alcanzar un rendimiento comparable al del modelo completo.



	precision	recall	f1-score	support
1	0.91	0.93	0.92	412
2	0.83	0.77	0.80	399
3	0.82	0.87	0.84	461
4	0.94	0.89	0.91	342
accuracy			0.87	1614
macro avg	0.87	0.87	0.87	1614
weighted avg	0.87	0.87	0.87	1614

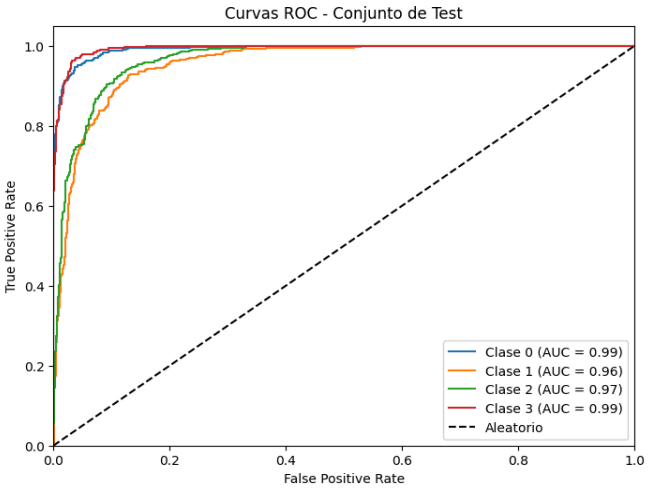
Finalmente vamos ver los resultados con el conjunto de Test:



Classification Report

	precision	recall	f1-score	support
1	0.94	0.94	0.94	3435
2	0.84	0.83	0.84	3081
3	0.86	0.89	0.87	3468
4	0.95	0.93	0.94	2923
accuracy			0.90	12907
macro avg	0.90	0.90	0.90	12907
weighted avg	0.90	0.90	0.90	12907

En las medidas de accuracy, recall y F1, dan resultados similares a las del conjunto train, indicando una buena generalización del modelo.



En la gráfica de la curva ROC se observa que las curvas se aproximan a la esquina superior izquierda, lo que indica un alto desempeño del modelo. Esto refleja que el modelo logra una alta tasa de verdaderos positivos, mientras mantiene baja la tasa de falsos positivos.

Respecto a la pérdida de cada clase, se mantienen similares a los del conjunto de entrenamiento y validación.

Poor	*Intermediate*	*Good*	*Excellent*
0.186	0.461	0.438	0.160

4. Conclusiones

En este trabajo hemos diseñado y evaluado de manera exitosa un modelo de aprendizaje profundo usando el módulo de Keras. En este caso, dado el dataset de jugadores del juego FIFA 19, hemos obtenido una clasificación de los jugadores en cuatro categorías que puntúan su calidad. Esto lo hemos conseguido mediante un proceso de experimentación iterativo, en donde se determinó que la regularización mediante Dropout, combinada con el optimizador Nadam con una tasa de aprendizaje de 0.0004 proporcionaba el equilibrio ideal. Una máxima mantenida a lo largo de todo este proceso fue tener siempre presente la complejidad del modelo, centrándonos en lograr el mejor resultado posible con el modelo más simple posible.

El modelo definitivo demostró una excelente capacidad de generalización, alcanzando un accuracy de 90% en el conjunto de test, y áreas bajo la curva ROC de entre 0.96 y 0.99. En cuanto al análisis de las pérdidas reveló que el clasificador identifica con mayor precisión las categorías extremas, debido a la dificultad inherente de establecer las fronteras de decisión claras entre las clases intermedias sin comprometer el rendimiento general.

Este proyecto nos ha demostrado la importancia de una metodología estructurada en el ajuste de los hiper parámetros, abordando primero el error de sesgo y posteriormente la varianza. Además, hemos podido comprobar de primera mano que una arquitectura neuronal más sencilla y eficiente puede igualar e incluso superar el rendimiento de las redes iniciales que por lo general suelen ser mucho más densas.

5. Referencias

- Transparencias y código aportados en Aprendizaje Automático II de la ETSIINF (UPM) realizadas por: daniel.manrique@upm.es y martin.molina@upm.es
- [1] The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks https://arxiv.org/abs/1803.03635